

3F7 Information Theory and Coding

Full Technical Report

Liu Zihe
zl559

University of Cambridge
December 6 2025

1 Introduction

Information theory provides the mathematical framework for quantifying information and establishing fundamental limits on data compression. Shannon’s Source Coding Theorem establishes the entropy $H(X)$ as the lower bound for the average code length of an independent and identically distributed (i.i.d.) source. While i.i.d. models are useful first approximations, real-world data sources—such as natural language—exhibit strong temporal dependencies (memory) and non-uniform distributions.

This report extends the investigation of the 3F7 short laboratory in two directions. First, we analyse **discrete stationary sources**, relaxing the i.i.d. assumption to model sources with memory. We investigate the *entropy rate* H_∞ of a stochastic process and the convergence properties of block entropies. Second, we explore **adaptive coding**, where the probability model is learned online instead of 2 separate phases (building the probability model from the complete text and encoding). We derive the Laplace estimator from Bayesian principles and evaluate the performance of adaptive arithmetic coding against static methods.

2 Overview of Short Lab

The short laboratory focused on compressing `hamlet.txt` (approx. 207kB using fixed-length ASCII encoding) using three classical techniques designed for i.i.d. sources.

Shannon-Fano Coding constructs prefix-free codes by assigning lengths $l_i = \lceil \log_2(1/p_i) \rceil$. While it guarantees unique decodability (via the Kraft inequality $\sum 2^{-l_i} \leq 1$), it is suboptimal because it ignores the combinatorial nature of packing the prefix-free code tree (codewords can still be placed at shorter leaf nodes, hence optimizing our encoding)

Huffman Coding is the optimal symbol-by-symbol prefix code. It builds a binary tree bottom-up by iteratively merging the two least probable symbols. Our experiments confirmed that its expected length L satisfies $H(X) \leq L < H(X) + 1$. However, its performance is constrained by the requirement that each symbol must use an integer number of bits.

Arithmetic Coding overcomes the integer constraint by encoding entire sequences as sub-intervals of $[0, 1)$. By mapping a sequence x^n to an interval of width $P(x^n)$, it achieves a code length $L \approx -\log_2 P(x^n)$ bits. For large n , the per-symbol length converges to the entropy $H(X)$. This method proved superior for longer blocks but required accurate probability models.

The limitation of all three methods in the short lab was the assumption that text is a memoryless source, ignoring the significant redundancy in English grammar and spelling.

3 Coding for Discrete Stationary Sources

3.1 Theoretical Background

We model natural language text as a sequence of random variables, specifically a *discrete stochastic process*, $\mathcal{X} = \{X_i\}_{i=1}^{\infty}$. We distinguish between two primary classes of sources:

Memoryless Sources (i.i.d.): The simplest model assumes that successive symbols are i.i.d. A source is *memoryless* if the joint probability of a sequence factorizes completely into the product of its components:

$$\mathbb{P}(X_1, \dots, X_n) = \prod_{i=1}^n \mathbb{P}(X_i). \quad (1)$$

For such a source, the information content is additive, and the entropy of a block is simply $H(X^n) = nH(X)$. The optimal code length per symbol is thus fixed at the marginal entropy $H(X)$. The fixed-length ASCII and standard Huffman coding (without blocking) implicitly assume this model.

Stationary Sources (Sources with Memory): Natural languages like English are not memoryless; the letter 'h' is much more likely to follow 't' than 'q'. To capture this, we relax the independence assumption while retaining time-invariance. A process is *strictly stationary* if the joint distribution of any subsequence is invariant to time shifts. That is, for any shift τ and block length n :

$$\mathbb{P}(X_1 = x_1, \dots, X_n = x_n) = \mathbb{P}(X_{1+\tau} = x_1, \dots, X_{n+\tau} = x_n). \quad (2)$$

Stationarity implies that marginal probabilities are constant $H(X_i) = H(X_1)$ and conditional probabilities depend only on the relative history, not the absolute time.

Entropy Rate: For a source with memory, the marginal entropy $H(X_1)$ overestimates the true information content because it ignores the predictability provided by the context. The fundamental limit is instead given by the *entropy rate* H_{∞} , defined as the asymptotic per-symbol entropy [1, Def 4.1]:

$$H_{\infty} = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, \dots, X_n). \quad (3)$$

This rate represents the *irreducible randomness* of the process after all past history is accounted for. To estimate H_{∞} , we define two sequences of block entropies:

$$H_n = \frac{1}{n} H(X_1, \dots, X_n) \quad (\text{Entropy per symbol of } n\text{-block}) \quad (4)$$

$$H_n^* = H(X_n | X_{n-1}, \dots, X_1) \quad (\text{Conditional entropy}) \quad (5)$$

3.2 Convergence of Entropy Rates (Theorem 1)

For stationary sources, these sequences converge to the entropy rate H_{∞} as $n \rightarrow \infty$. We provide proofs for the properties stated in the FTR handout, referencing Cover & Thomas [1, Chap 4].

Statement 1: $H_{n+1}^* \leq H_n^*$ (Conditional entropy is non-increasing).

Proof:

$$\begin{aligned} H_{n+1}^* &= H(X_{n+1} | X_n, \dots, X_2, X_1) \\ &\leq H(X_{n+1} | X_n, \dots, X_2) \quad (\text{Conditioning reduces entropy}) \\ &= H(X_n | X_{n-1}, \dots, X_1) \quad (\text{By Stationarity}) \\ &= H_n^* \quad \square \end{aligned}$$

Statement 2: $H_{n+1} \leq H_n$ (Per-symbol entropy is non-increasing).

Proof: By the chain rule, $H(X_1, \dots, X_{n+1}) = H(X_1, \dots, X_n) + H(X_{n+1}|X_1, \dots, X_n)$. Then

$$H_{n+1} = \frac{H(X_1, \dots, X_{n+1})}{n+1} = \frac{nH_n + H_{n+1}^*}{n+1} \quad (6)$$

Since $H_{n+1}^* \leq H_n^*$ (from Statement 1), $H_n^* \leq H_n$ (from statement 3), then $H_{n+1}^* \leq H_n^* \leq H_n$ where H_n is the average of joint entropy across the n terms. Adding a term H_{n+1}^* that is smaller than or equal to the previous average must reduce (or maintain) the new average. Thus $H_{n+1} \leq H_n$. $\square$

Statement 3: $H_n^* \leq H_n$.

Proof: From the Chain Rule for entropy,

$$H(X_1, \dots, X_n) = \sum_{i=1}^n H(X_i|X_{i-1}, \dots, X_1). \quad (7)$$

Dividing by n :

$$H_n = \frac{1}{n} \sum_{i=1}^n H_i^*. \quad (8)$$

Since H_i^* is a non-increasing sequence (Statement 1), the current term H_n^* is less than or equal to all preceding terms H_i^* for $i < n$. Therefore, the average H_n must be greater than or equal to the minimum term H_n^* . $\square$

Statement 4: $\lim_{n \rightarrow \infty} H_n = \lim_{n \rightarrow \infty} H_n^* = H_\infty$.

Proof: Since H_n^* is non-increasing (Statement 1) and bounded below by 0 (entropy is non-negative), the limit $\lim_{n \rightarrow \infty} H_n^* = H'$ exists. Since H_n is the arithmetic mean (Cesàro mean) of the sequence H_n^* as $n \rightarrow \infty$, by the *Cesàro Mean Theorem* (if $a_n \rightarrow a$, then $\frac{1}{n} \sum a_i \rightarrow a$), the sequence of averages converges to the same limit [1, Thm 4.2.1]. This limit is defined as the *entropy rate* H_∞ . $\square$

3.3 Experimental Results: Entropy Rate Convergence

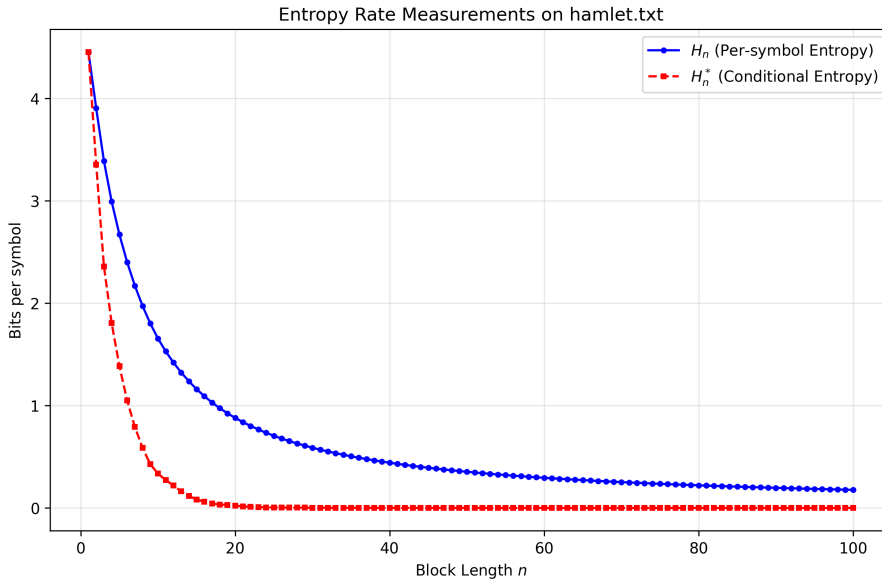


Figure 1: Entropy rate measurements on `hamlet.txt`. The estimates converge to 0 for large n due to data sparsity, not because the text is deterministic.

We measured the block entropy per symbol, $H_n = \frac{1}{n}H(X^n)$, for $n = 1$ to 100. As shown in Fig. 1, estimates for small n ($n = 120$) decrease quickly, correctly reflecting that English text is a source with memory; knowledge of previous characters reduces the uncertainty of the current one. However, for $n \geq 4$, the estimate continues to decay toward zero. The true entropy rate of English (H_∞) is surely not zero. Rather, this decay is a statistical artifact of a maximum likelihood estimator operating in a high-dimensional regime with insufficient data (the “curse of dimensionality”).

Let N be the total length of the text (≈ 200000) and $|\mathcal{X}|$ be the alphabet size (≈ 70). The number of possible sequences $|\mathcal{X}|^n$ grows exponentially. When $|\mathcal{X}|^n \gg N$, the vast majority of observed n -tuples appear only once. In this “sparse” regime, the empirical distribution $\hat{p}(x^n)$ becomes uniform over the N observed unique sequences, with probability $1/N$ for each. The estimated block entropy becomes:

$$\hat{H}(X^n) = -\sum_{i=1}^N \frac{1}{N} \log_2 \left(\frac{1}{N} \right) = \log_2 N$$

Consequently, the estimated entropy rate per symbol behaves as:

$$\hat{h}_n = \frac{\hat{H}(X^n)}{n} \approx \frac{\log_2 N}{n}$$

As $n \rightarrow \infty$, this term vanishes ($\lim_{n \rightarrow \infty} \frac{\text{const}}{n} = 0$). This proves that the observed decay is a bias of the estimator, not a property of the source. Measurement of H_∞ is impossible without a sample size N that scales as $|\mathcal{X}|^n$.

3.4 Implications for Static Coding

The failure of the entropy estimator highlights a critical limitation in static source coding (e.g., Static Huffman or Static Arithmetic). To approach the entropy rate of a stationary source, we must exploit context. In a static framework, this implies constructing a code for blocks of length n (or using a conditional probability model of order n).

However, for a static code, the encoder must transmit the model (the codebook or frequency table) to the decoder as side information. The size of this model is proportional to the number of possible contexts $|\mathcal{X}|^n$.

- **Exploding Overhead:** As we increase n to capture more dependencies, the model overhead grows exponentially (70^n), quickly exceeding the size of the data itself.
- **The Trade-off:** There is an optimal finite n that balances the reduction in coding rate (better compression) against the increase in model overhead. We cannot simply let $n \rightarrow \infty$.

3.5 Practical Techniques for Large n : Lempel-Ziv

The handout asks if there is a technique where n can be made large to approach the entropy rate. The answer is **universal source coding**, specifically **Lempel-Ziv (LZ)** algorithms (e.g., LZ77, LZ78).

Unlike Huffman or Arithmetic coding, LZ does not rely on explicit probability estimates or frequency tables. Instead, it operates by replacing variable-length phrases in the source with pointers to previous occurrences of those phrases.

- **Mechanism:** As the algorithm processes more text, it builds a dictionary of longer and longer substrings. A pointer (d, l) representing a match of length l at distance d effectively exploits a context of length l .

- **Asymptotic Optimality:** As the file length $N \rightarrow \infty$, the average match length l grows. This allows LZ to implicitly exploit dependencies of arbitrary length (n) without the overhead of transmitting an exponential probability table. It is asymptotically optimal for stationary ergodic sources, meaning it approaches H_∞ as $N \rightarrow \infty$.

4 Adaptive Coding

4.1 Bayesian Derivation of the Laplace Estimator

In adaptive coding, we face the “zero-frequency problem”: if a symbol x has not yet appeared in the stream, its Maximum Likelihood Estimate (MLE) is $P(x) = 0$. This is catastrophic for arithmetic coding. To solve this, we must assign probability mass to unseen events. We derive the solution using Bayesian inference, which treats the unknown probabilities not as fixed values, but as random variables describing our uncertainty [3, p. 97].

4.1.1 The Beta Prior and Posterior

Consider a binary version of the problem where we simply estimate the probability θ that the next symbol is a specific character ‘x’. We start with **Bayes’ Theorem**:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)} \quad (9)$$

where D is the observed data (sequence of symbols), $P(\theta)$ is the *prior* belief, and $P(D|\theta)$ is the *likelihood* of the data.

1. The Prior (Before Data): Since we assume no prior knowledge about the text, we must choose a prior that represents total ignorance. For a probability parameter $\theta \in [0, 1]$, the natural choice is the **Uniform Distribution**, also known as the *Beta*(1, 1) distribution [2, Ch 3]. In the Bayesian framework, this corresponds to a prior which expresses no initial preference for one parameter set over another:

$$P(\theta) = 1 \quad \text{for } 0 \leq \theta \leq 1. \quad (10)$$

This implies that before we start reading, we consider every possible value for the probability of ‘x’ to be equally likely.

2. The Likelihood (The Data): Suppose we observe a stream of length n containing n_x occurrences of ‘x’ and n_y occurrences of other symbols ($n_y = n - n_x$). Assuming independence given θ , the probability of seeing this specific sequence is:

$$P(D|\theta) = \theta^{n_x} (1 - \theta)^{n_y}. \quad (11)$$

3. The Posterior (Updated Belief): Multiplying the prior and likelihood gives us the posterior distribution (proportionality ignores the normalizing constant $P(D)$):

$$P(\theta|D) \propto \theta^{n_x} (1 - \theta)^{n_y} \cdot 1. \quad (12)$$

This resulting distribution is a Beta distribution with parameters $(n_x + 1, n_y + 1)$. As n grows, this curve becomes a sharp peak around the empirical frequency n_x/n , representing our growing confidence.

4.1.2 The Rule of Succession (Prediction)

To encode the *next* symbol X_{n+1} , we do not simply use the most probable value of θ (the peak). Instead, we must average over *all possible* values of θ , weighted by their posterior probability. This process is called **marginalization**:

$$P(X_{n+1} = x|D) = \int_0^1 P(X_{n+1} = x|\theta)P(\theta|D) d\theta. \quad (13)$$

Since $P(X_{n+1} = x|\theta) = \theta$ by definition, this integral calculates the **expected value** (mean) of the posterior Beta distribution. This corresponds to the Minimum Mean-Squared Error (MMSE) estimator in Bayesian theory, which is defined as the expectation of the posterior [3, p. 129]. The mean of a $Beta(\alpha, \beta)$ distribution is $\frac{\alpha}{\alpha+\beta}$ [2]. Substituting our posterior parameters $\alpha = n_x + 1$ and $\beta = n_y + 1$:

$$\hat{p}(x) = \mathbb{E}[\theta|D] = \frac{n_x + 1}{(n_x + 1) + (n_y + 1)} = \frac{n_x + 1}{n + 2}. \quad (14)$$

This result is the famous **Laplace's Rule of Succession**. It provides a rigorous justification for "add-one smoothing": we are not arbitrarily adding 1, but rather including the "pseudocounts" from our uniform prior.

4.1.3 Generalization to Text (Dirichlet Distribution)

For an alphabet of size K (e.g., $K = 256$ for ASCII), the Beta distribution generalizes to the **Dirichlet distribution** [2, Ch 3]. The uniform prior corresponds to a Dirichlet distribution with parameter vector $\alpha = (1, 1, \dots, 1)$. The predictive probability for any symbol x becomes:

$$\hat{p}(x) = \frac{n_x + 1}{\sum_{i=1}^K (n_i + 1)} = \frac{n_x + 1}{n + K}. \quad (15)$$

This is the estimator we implemented in the laboratory. It ensures that every symbol always has a non-zero probability of at least $1/(n + K)$, preventing the infinite code length problem while converging to the true frequency n_x/n as $n \rightarrow \infty$.

4.2 Experimental Performance

We implemented an adaptive arithmetic coder using this estimator.

Convergence: Fig. 2 shows the running average code length L/n converging to the static entropy H_1 . The "learning penalty" (excess code length) decreases as $O(1/n)$, consistent with universal coding theory [1, Sec 13.2].

Bias Parameter δ : We varied δ (Fig. 3) and found the optimal range $\delta \in [0.5, 1.0]$.

- Small δ (< 0.1): Overfitting. Unseen symbols have near-zero probability, causing massive penalties.
- Large δ (> 5): Underfitting. The strong prior washes out the observed data, slowing adaptation.

Adaptive vs. Non-Adaptive: Fig. 4 compares total bits. Non-adaptive coding requires overhead to store the frequency table ($\approx K \log n$ bits). Adaptive coding has no header but higher initial redundancy. The crossover point for Hamlet is at $n \approx 50,000$ symbols. For files shorter than this, adaptive coding is more efficient.

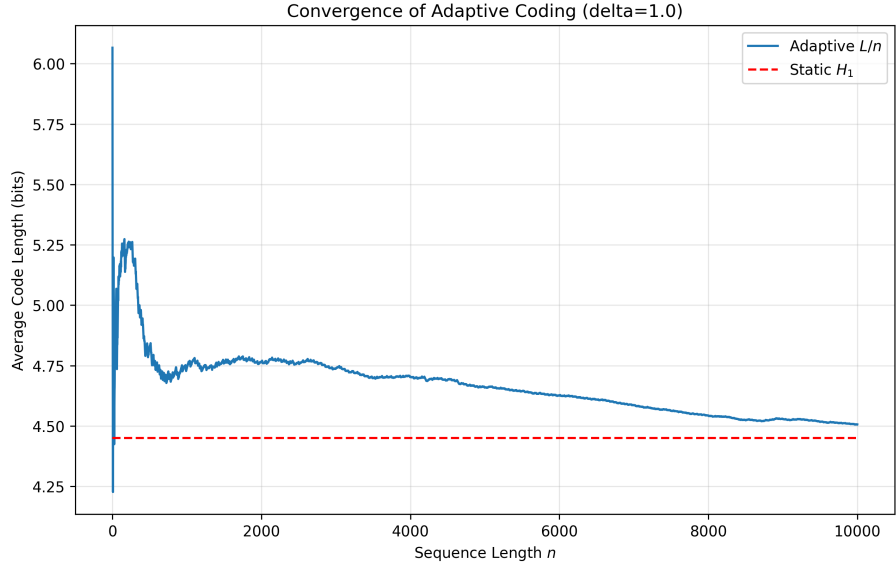


Figure 2: Convergence of adaptive coding rate (L/n) to the empirical entropy (H_1) for $\delta = 1$. The overhead vanishes as the model "learns" the source statistics.

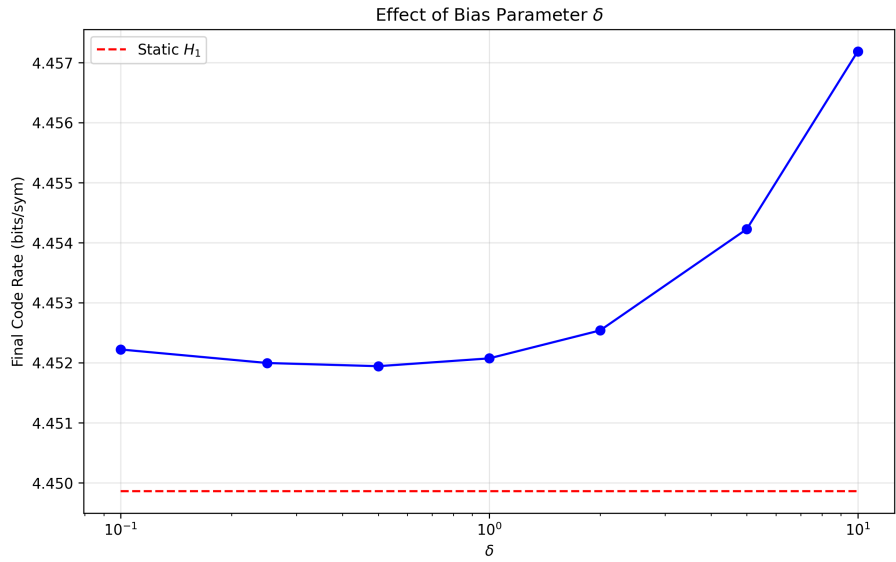


Figure 3: The effect of the bias parameter δ . The minimum code length is achieved around $\delta \approx 1.0$, corresponding to the Laplace estimator.

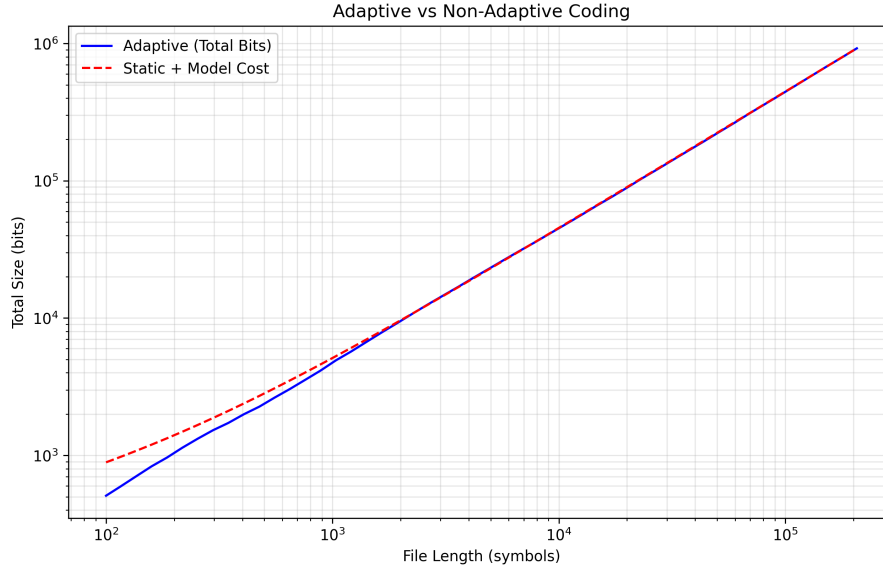


Figure 4: Total bits required for Adaptive vs Non-Adaptive coding. The crossover at $n \approx 50,000$ highlights the efficiency of adaptive coding for short streams.

5 Conclusion

This report demonstrated that while English text is a stationary source with memory, exploiting this theoretically (via H_∞) is hindered by sparse data. Adaptive coding offers a robust solution, allowing compression to approach the source entropy without prior knowledge of statistics. By deriving the Laplace estimator from Bayesian principles, we showed that adding a “pseudo-count” of 1 is not heuristic but the optimal strategy under a uniform prior assumption. This principle underpins modern compressors like LZ77 and prediction models in Large Language Models.

References

- [1] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.
- [2] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.
- [3] S. Godsill, *Estimation Theory and Inference Methods*, Lecture Notes (3F3), University of Cambridge, 2023